01_getting_started_ja

NEORV32 入門（日本語訳）

原文: `neorv32/rtl/test_setups/README.md` と `neorv32/docs/userguide/` の次の 4 章

- general_hw_setup.adoc
- sw_toolchain_setup.adoc
- application_program_compilation.adoc
- installing_an_executable.adoc

翻訳対象のバージョン: v1.13.7-23-g7d483648

「訳注」は原文にはない補足です（Ti375 で使う場合の注意など）。

第 0 部 テストセットアップ（rtl/test_setups/README.md）

このフォルダには、とても簡単なテストセットアップが入っています。NEORV32 を初めて使う人が、最小構成の NEORV32 SoC を組むためのものです。これらのセットアップは、NEORV32 ユーザーガイドでも使われています。

最初は話を簡単にするため、最小限の構成になっています。CPU の ISA 拡張、性能オプション、周辺モジュールを追加したい場合は、対応する設定ジェネリック（configuration generics）を指定すれば有効にできます。

クロックとリセット

どのテストセットアップも、外部クロック（`clk_i`）と、外部のアクティブ Low のリセット（`rstn_i`）が必要です。

設定ジェネリック

どのセットアップにも、FPGA やボードに合わせて変更できる（または変更すべき）基本的なジェネリックが 3 つあります。

- クロック周波数（Hz）は、`CLOCK_SPEED` ジェネリックで**必ず指定する必要があります**。クロック源に合わせてください。
- プロセッサ内部の命令メモリ（IMEM）のサイズは、`IMEM_SIZE` ジェネリックで変更**できます**。
- プロセッサ内部のデータメモリ（DMEM）のサイズは、`DMEM_SIZE` ジェネリックで変更**できます**。

> **警告**: メモリサイズを変更した場合は、NEORV32 のリンカスクリプトも合わせて変更する必要があるかもしれません。

訳注: README では `CLOCK_SPEED` と書かれていますが、ユーザーガイドや VHDL のファイルでは `CLOCK_FREQUENCY` となっています。実際のファイルで名前を確認してください。

neorv32_test_setup_approm.vhd

`rv32imc_Zicsr_Zicntr` の CPU に、次のものを組み合わせた構成です。

- 16kB の IMEM（初期値入りの ROM として実装）
- 8kB の DMEM
- 外部の 8 本の信号（`gpio_o`）を駆動する **GPIO** モジュール
- タイマ割り込みを作る **CLINT** モジュール

「ダイレクトブート」構成です。ソフトウェアは合成のときに、ビットストリーム経由でプロセッサ内部の IMEM に直接「組み込まれ」ます。

> ヒント: ユーザーガイドの「実行ファイルをメモリに直接組み込む」（第 4 部）を参照。

neorv32_test_setup_bootloader.vhd

`rv32imc_Zicsr_Zicntr` の CPU に、次のものを組み合わせた構成です。

- 16kB の IMEM（RAM として実装）
- 8kB の DMEM
- 外部の 8 本の信号（`gpio_o`）を駆動する **GPIO** モジュール
- タイマ割り込みを作る **CLINT** モジュール
- ブートローダやアプリとシリアル端末でやりとりするための **UART0**（`uart0_txd_o`、`uart0_rxd_i`）

「インダイレクトブート」構成です。ブートローダとシリアル端末を使って、ソフトウェアをいつでもアップロードして実行できます。

> ヒント: ユーザーガイドの「UART 経由でバイナリ実行イメージをアップロードして起動する」を参照。

neorv32_test_setup_on_chip_debugger.vhd

`rv32imc_Zicsr_Zicntr_Zifencei` の CPU に、次のものを組み合わせた構成です。

- 16kB の IMEM（RAM）
- 8kB の DMEM
- GPIO（8 本）
- CLINT
- UART0

- RISC-V 互換のオンチップデバッガ（**OCD**）。標準的な JTAG インターフェイス（`jtag_*`）からアクセスする

インダイレクトブート構成です。ソフトウェアは、ブートローダとシリアル端末からアップロードして実行できます。オンチップデバッガからも同じことができます。

> ヒント: ユーザーガイドの「オンチップデバッガを使ったデバッグ」を参照。

訳注: Sapphire のときは、OpenOCD で JTAG から RAM に書き込みました。NEORV32 では、UART のブートローダから書き込む方法（bootloader 版）が一番簡単です。JTAG を使う場合は、Ti375 の JTAG User TAP と NEORV32 の `jtag_*` をつなぐ工夫が必要になります。

第 1 部 ハードウェアの基本セットアップ（general_hw_setup.adoc）

このガイドでは、NEORV32 のプロジェクトを、シミュレーション用または合成用にゼロから組む基本手順を説明します。最初は話を簡単にするため、プロセッサの簡単なテスト用「SoC」構成を例に使います。この構成は、プロセッサを初めて試す・評価するためのものです。

NEORV32 プロジェクトには、あらかじめ設定された最小限のテストセットアップが 3 つ、`rtl/test_setups` に用意されています。これらはごく基本的なプロセッサと CPU の機能だけを実装したもので、主な違いはブートの方法です。

（図: NEORV32 の「hello world」テストセットアップ `rtl/test_setups/neorv32_test_setup_bootloader.vhd`。`docs/figures/neorv32_test_setup.png`）

1. 使いたい FPGA / ASIC / シミュレータの EDA ツールで、新しいプロジェクトを作ります。
2. プロジェクトの `rtl/core` フォルダにある VHDL ファイルを、すべてプロジェクトに追加します。これらの RTL ファイルは、すべて `neorv32` という名前の新しいライブラリに入れてください。ツールにライブラリ名を入力する欄がない場合は、追加した RTL ファイルの「プロパティ」メニューを確認してください。

> **ヒント: コンパイル順とファイルリスト** ツールによっては（Lattice Radiant など）、依存関係を把握するために VHDL ファイルのコンパイル順を手動で指定する必要があります。`rtl` フォルダには、必要な HDL ファイルを推奨のコンパイル順に並べたファイルリストがあります。

3. `rtl/core/neorv32_top.vhd` が NEORV32 プロセッサのトップエンティティで、ユーザーのプロジェクトの中でインスタンス化して使えます。ただしこのチュートリアルでは、`rtl/test_setups` にあるテストセットアップのどれかをトップエンティティとして使います。

> **重要: NEORV32 の VHDL パッケージ** プロセッサをインスタンス化するときは、`neorv32` パッケージを読み込んでください。設計ユニットに `library neorv32;` と `use neorv32.neorv32_package.all;` を追加します。

4. 使いたいテストセットアップもプロジェクトに追加し、それを**トップエンティティ**に指定します。
5. テストセットアップのエンティティには、最小限の設定ジェネリックがあります。FPGA やボードに合わせて変更が必要なことがあります。

```
generic (
  -- adapt these for your setup --
  CLOCK_FREQUENCY : natural := 100000000; -- (1)
  IMEM_SIZE       : natural := 16*1024;   -- (2)
  DMEM_SIZE       : natural := 8*1024     -- (3)
);
```

- (1) `clk_i` 信号のクロック周波数（Hz）
- (2) 内部命令メモリのデフォルトサイズ: 16kB
- (3) 内部データメモリのデフォルトサイズ: 8kB

6. FPGA の資源が足りない場合は、メモリサイズ（`IMEM_SIZE` と `DMEM_SIZE`）を変更できます。
7. トップのクロック入力（`clk_i`）の実際の周波数（Hz）を、`CLOCK_FREQUENCY` ジェネリックで指定する必要があります。
8. テストセットアップのトップエンティティの信号を、FPGA ボードの対応するピンに割り当てます。信号はすべて、各テストセットアップのエンティティ宣言に書かれています。例:

```
port (
  -- Global control --
  clk_i       : in  std_ulogic; -- global clock, rising edge
  rstn_i      : in  std_ulogic; -- global reset, low-active, async
  -- GPIO --
  gpio_o      : out std_ulogic_vector(7 downto 0); -- parallel output
  -- UART0 --
  uart0_txd_o : out std_ulogic; -- UART0 send data
  uart0_rxd_i : in  std_ulogic  -- UART0 receive data
);
```

> **重要: 信号の極性** ボードの入出力の極性が逆の場合は、インバータを追加する必要があります。たとえば、リセット信号 `rstn_i` はデフォルトでアクティブ Low です。`gpio_o` につなぐ LED は、デフォルトでアクティブ High です。

9. クロック入力 `clk_i` をクロック源につなぎ、リセット `rstn_i` をボードのボタンにつなぎます。ボタンがアクティブ Low か High かを確認してください。プロセッサのリセットは**アクティブ Low** なので、入力を反転する必要があるかもしれません。
10. できれば、GPIO 出力 `gpio_o` の少なくともビット 0 を LED につなぎます（上の「信号の極性」を参照）。
11. UART を使うテストセットアップの場合は、UART の信号 `uart0_txd_o` と `uart0_rxd_i` を、ホスト側のインターフェイス（USB-UART 変換器など）につなぎます。

12. オンチップデバッガ版のセットアップの場合は、プロセッサの JTAG 信号 `jtag_*` を適切な JTAG アダプタにつなぎます。
13. プロジェクトの HDL コンパイル（合成、マッピング、配置、配線、ビットストリーム生成）を実行します。
14. 生成したビットストリームを FPGA に書き込み、リセットにつないだボタンを押します。
15. `gpio_o` につないだ LED が点滅するはずです。

訳注（Ti375 の場合）:

- クロック: Sapphire と同じく、PLL の 100MHz 出力を `clk_i` に使えます。
- リセット: SW3（GPIOB_P_31）は押すと 0 なので、アクティブ Low の `rstn_i` にそのままつなげます。PLL のロックも考えるなら、`rstn_i <= rst_n and pll_locked` のようにします。
- UART: Sapphire と同じピン（TX: GPIOL_24、RX: GPIOL_20）が使えます。
- LED: `gpio_o` は 8 本ですが、ボードの LED は 4 本（LED1〜4）なので、下位 4 ビットをつなぎます。
- 手順 2 のライブラリ名 `neorv32` の設定方法を、Efinity で確認してください。

第 2 部 ソフトウェアのツールチェーン準備（sw_toolchain_setup.adoc）

NEORV32 用の実行ファイルをコンパイル（およびデバッグ）するには、RISC-V 対応のツールチェーンが必要です。プロジェクトのソフトウェアフレームワークは、デフォルトで GNU C コンパイラの RISC-V 版「RISC-V GCC」を使います。ツールチェーンを手に入れる方法は、基本的に 2 つあります。

1. RISC-V GNU ツールチェーンをダウンロードして、自分で**ビルド**する。
2. **ビルド済み**のツールチェーンをダウンロードして、**インストール**する。

> **注: デフォルトの GCC 接頭辞** このプロジェクトのデフォルトのツールチェーン接頭辞は `riscv-none-elf-` です。この接頭辞は、NEORV32 のアプリ用 makefile の `RISCV_PREFIX` 変数で変更できます。

ツールチェーンの要件

1. 32 ビットアーキテクチャ用のコード（`mabi=rv32`）を出力できること。
2. 組み込み用の C 標準ライブラリ（たとえば Newlib）を使うこと。

> **重要: ライブラリと ISA の注意** `--with-arch=rv32imc` でビルドしたツールチェーンでは、ライブラリのコード（C 標準ライブラリなど）がすべて、圧縮命令（`C`）と乗除算命令（`M`）を使ってコンパイルされています。そのため、これらの ISA 拡張に対応していない CPU では、このビルド済みライブラリのコードを（エミュレーションなしでは）実行**できません**。

ツールチェーンをゼロからビルドする

公式の RISC-V GCC の GitHub リポジトリ（https://github.com/riscv-collab/riscv-gnu-toolchain ）に、ツールチェーンをゼロからビルドする手順があります。

```
$ git clone https://github.com/riscv/riscv-gnu-toolchain
$ cd riscv-gnu-toolchain
riscv-gnu-toolchain$ ./configure --prefix=/opt/riscv --with-arch=rv32i --with-abi=ilp32
riscv-gnu-toolchain$ make
```

（この例は最小 ISA の `rv32i` 用の設定です）

この設定でビルドすると、接頭辞（GCC トリプレット）が `riscv32-unknown-elf-` のツールチェーンになります。そのため、NEORV32 の makefile を実行する前に、`RISCV_PREFIX` 変数をそれに合わせて変更する必要があります。

ビルド済みツールチェーンをダウンロードしてインストールする

ビルド済みのツールチェーンを使うこともできます。OS のパッケージマネージャによっては、組み込み用の RISC-V GCC ツールチェーンが用意されています。ただし著者のおすすめは、xPack プロジェクトが提供するツールチェーン（MIT ライセンス）です: https://github.com/xpack-dev-tools/riscv-none-elf-gcc-xpack

ツールチェーンの組み込み

選んだツールチェーンを NEORV32 のソフトウェアフレームワークで使うには、ツールチェーンの実行ファイルをシステムのパス（`PATH` 環境変数など）に追加して、シェルから使えるようにします。そのために、ツールチェーンの `bin` フォルダの絶対パスを `PATH` 変数に追加します。

```
$ export PATH=$PATH:/opt/riscv/bin
```

> ヒント: このコマンドを `.bashrc` などに追加しておけば、ターミナルを開くたびに自動で RISC-V ツールチェーンが使えるようになります。

正しく動くか確かめるには、NEORV32 の `sw/example` フォルダにあるサンプルのどれかに移動して、次のコマンドを実行します。

```
neorv32/sw/example/demo_blink_led$ make check
```

NEORV32 の実行ファイルを作るのに必要なツールが、すべてテストされます。ログの最後に「Toolchain check OK」と出れば、すべて正常です。

訳注: Efinix の IDE に付属している GCC は、xPack 版の `riscv-none-elf-gcc` 13.4.0 です。著者のおすすめと同じものなので、`.bashrc` に追加済みの PATH のまま使えるはずです。まず `make check` で確認してください。

第 3 部 アプリケーションのコンパイル（application_program_compilation.adoc）

このガイドでは、NEORV32 プロセッサ用の C のアプリケーションをコンパイルする方法を説明します。

1. ターミナルを開き、プロジェクトのサンプルプログラムのどれかに移動します。たとえば、簡単なサンプル `sw/example/demo_blink_led` に移動します。このプログラムは GPIO コントローラを使って、`gpio_o` 出力の下位 8 ビットに 8 ビットのカウンタを表示します。
2. プロジェクトをコンパイルして実行ファイルを作るには、次のコマンドを実行するだけです。

```
neorv32/sw/example/demo_blink_led$ make clean_all exe
```

3. `clean` ターゲットは、すべてを確実にビルドし直すために使います。
4. `exe` ターゲットは、アプリのソースと、読み込んだライブラリをすべてコンパイルしてリンクします。最後に ELF ファイル（`main.elf`）ができます。NEORV32 イメージジェネレータ（`sw/image_gen`）がこのファイルを受け取り、最終的な実行ファイル（`neorv32_exe.bin`）を作ります。makefile は、メモリ使用量と実行ファイルのサイズを表示します。

```
neorv32/sw/example/demo_blink_led$ make clean_all exe
Memory utilization:
   text    data     bss     dec     hex filename
   1004       0       0    1004     3ec main.elf
Compiling ../../../sw/image_gen/image_gen
Executable (neorv32_exe.bin) size in bytes:
1016
```

> **注: ビルドの生成物** 中間的な生成物（オブジェクトファイルやバイナリなど）は、プロジェクト内に新しく作られる `build` フォルダに置かれます。メインの ELF と実際の実行ファイルは、プロジェクトのルートフォルダに置かれます。

5. `exe` ターゲットで、今いるフォルダに実際の実行ファイル `neorv32_exe.bin` ができました。これは、内蔵のブートローダを使ってプロセッサにアップロードできる状態です。また、`main.elf` はオンチップデバッガを使ってアップロードできます。

第 4 部 実行ファイルをメモリに直接組み込む（installing_an_executable.adoc）

アプリケーションを、内部の命令メモリ（IMEM）に不揮発のイメージとして組み込むことができます。こうすると、リセット直後にそのアプリが実行されます。この方法では、IMEM は初期値入りの ROM として実装され、FPGA にビットストリームを書き込むときに初期化されます。

1. プロセッサのブートモード `BOOT_MODE_SELECT` を「Boot IMEM Image」（値 2）に設定します。このブートモードには、プロセッサ内部の命令メモリ（IMEM）が必要なので、有効になっていること（`IMEM_EN`）を確認してください。

```
BOOT_MODE_SELECT => 2,    -- boot from pre-initialized IMEM-ROM
IMEM_EN          => true, -- implement processor-internal instruction memory
```

2. デフォルトの IMEM 初期化ファイル（`rtl/core/neorv32_imem_image.vhd`）を作り直して組み込み、実際のアプリのイメージが入るようにします。

```
neorv32/sw/example/demo_blink_led$ make clean_all image install
Memory utilization:
   text    data     bss     dec     hex filename
  22192    1352    4216   27760    6c70 main.elf
Compiling image generator...
Generating neorv32_imem_image.vhd
Installing application image to ../../../rtl/core/neorv32_imem_image.vhd
```

3. FPGA の合成をやり直し、ビットストリームを書き込みます。これでアプリのコードはプロセッサの IMEM に不揮発で入っていて、リセット直後に実行されます。

訳注: この方法は、Sapphire でいうと「RAM の初期値にプログラムを入れておく」やり方です。プログラムを変えるたびに FPGA のビルドからやり直しになります。最初の動作確認には、UART ブートローダ版（bootloader）のほうが便利です。